

Manual de Pruebas Automatizadas, Suites de Integración y Benchmarks

Este documento describe la arquitectura de pruebas automatizadas de **Ateneo+**, los comandos para su ejecución bajo demanda y la interpretación de los artefactos generados para el artículo científico.

1. Pirámide de Pruebas Automatizadas

El sistema organiza sus pruebas en 4 niveles complementarios:

NIVEL 4: BENCHMARK CUANTITATIVO EXPERIMENTAL PARA PAPER (IR + LLM)

- run_metrics.py (25 casos, Hit@1, MRR@5, NDCG@5, latencias P50/P95)
- run_ablation_study.py (Ablación BM25 vs Dense Base vs RAG Híbrido)
- run_faithfulness_benchmark.py (Fidelidad normativa y anti-alucinación)

NIVEL 3: PRUEBAS DE MODELADO PSICOMÉTRICO Y CURRÍCULO ADAPTATIVO

- test_adaptive_curriculum.py (Topología KST 7 nodos, BKT, ZDP)
- test_paper_differentiators.py (Métricas IBF de cohorte y alertas)
- pilot_study_analyzer.py (Ganancia de aprendizaje de Hake y Wilcoxon)

NIVEL 2: PRUEBAS DE INTEGRACIÓN HTTP DE API REST (FASTAPI TESTCLIENT)

- test_api_endpoints.py (Rutas /auth, /cases, /history, /adaptive, /rooms)

NIVEL 1: VALIDACIÓN DE RECUPERACIÓN DETERMINISTA Y GENERADORES BINARIOS

- test_multimodal_and_cases.py (12 casos ChromaDB, PDF ReportLab SHA-256)

2. Catálogo de Scripts de Prueba Disponibles

2.1 Orquestador Maestro Rápido (backend/tests/run_all_tests.py)

Ejecuta de forma secuencial las suites de modelado adaptativo, diferenciadores y análisis de ganancia de Hake:

```
docker compose exec backend python tests/run_all_tests.py
```

2.2 Suite de Currículo Adaptativo (backend/tests/test_adaptive_curriculum.py)

Valida la topología del grafo KST, la actualización de probabilidades de dominio BKT ($P(L_{t+1})$) ante aciertos/fallos, la detección de la Zona de Desarrollo Próximo y los endpoints `/api/adaptive/*`:

```
docker compose exec backend python tests/test_adaptive_curriculum.py
```

2.3 Suite de Diferenciadores Científicos

([backend/tests/test_paper_differentiators.py](#))

Valida el algoritmo de cálculo de Faithfulness Score, el Índice de Brecha Formativa (IBF) por cohorte y los endpoints B2B:

```
docker compose exec backend python tests/test_paper_differentiators.py
```

2.4 Analizador de Ganancia de Aprendizaje ([backend/tests/pilot_study_analyzer.py](#))

Procesa el dataset piloto ([resultados_pilot.csv](#)), calcula la ganancia normalizada de Hake (\$g\$) y el estadístico \$t\$ pareado, y exporta la Tabla IV en LaTeX:

```
docker compose exec backend python tests/pilot_study_analyzer.py
```

2.5 Suite de Integración de Endpoints HTTP ([backend/tests/test_api_endpoints.py](#))

Ejecuta una batería completa de pruebas sobre las rutas de FastAPI mediante `TestClient`:

```
docker compose exec backend python tests/test_api_endpoints.py
```

2.6 Validación de Casos Clínicos y Multimodal

([backend/tests/test_multimodal_and_cases.py](#))

Verifica la recuperación exacta de los 12 casos del catálogo contra ChromaDB, la generación de PDFs institucionales y la evaluación multi-imagen con Gemini Vision:

```
docker compose exec backend python tests/test_multimodal_and_cases.py
```

2.7 Benchmark Cuantitativo del Paper ([backend/tests/run_metrics.py](#))

Ejecuta la evaluación experimental completa sobre los 25 casos In-Distribution y Out-of-Distribution, generando la Tabla I en LaTeX ([tabla_resultados_paper.tex](#)):

```
docker compose exec backend python tests/run_metrics.py
```

3. Artefactos LaTeX Generados para el Paper

Archivo Generado	Tabla del Paper	Métrica Central Reportada
docs/tabla_resultados_paper.tex	Tabla I	Rendimiento IR (Hit@1, Hit@5, MRR@5, NDCG@5, Latencias)
docs/tabla_ablacion_paper.tex	Tabla II	Estudio de Ablación Arquitectónica (Sparse vs Dense vs Híbrido)
docs/tabla_faithfulness_paper.tex	Tabla III	Fidelidad Normativa RAG (Faithfulness Score vs Baseline)
docs/tabla_pilot_study_paper.tex	Tabla IV	Ganancia de Aprendizaje de Hake (\$g\$) Pre-Test vs Post-Test

4. Verificación de Compilación de Frontend

Para compilar y convalidar la ausencia de errores de sintaxis, hooks o tipos en el cliente React 18:

```
docker compose exec frontend npm run build
```

Salida esperada: 1,600+ módulos transformados, 0 errores de compilación y generación de service worker PWA.